

04-高精度

一、本阶段目标

高精度专题的核心目标是：

- 理解为什么普通整数类型不够用
- 掌握用字符串存大整数的方法
- 理解高精度本质是在模拟小学竖式
- 掌握高精度加法、减法、乘法
- 熟悉进位、借位、前导 0 处理
- 能独立完成基础高精度题

高精度不是一种很玄的算法。

它的本质是：

- 1 计算机不能直接存很大的数，所以我们把每一位拆开，自己模拟计算过程。

二、为什么需要高精度？

C++ 常见整数范围：

- 1 int: 大约 2×10^9
- 2 long long: 大约 9×10^{18}

如果题目给出：

- 1 一个有 100 位、1000 位，甚至更多位数的整数

那么 long long 就装不下。

这时需要用：

- 1 string

或者：

```
1 vector<int>
```

来一位一位存储数字。

三、高精度的核心思想

高精度本质是：

```
1 模拟小学竖式。
```

比如：

```
1    123
2  + 456
3  -----
4    579
```

我们真正做的是：

```
1  个位加个位
2  十位加十位
3  百位加百位
4  处理进位
```

程序里也是一样。

四、为什么通常从低位开始算？

加法、减法、乘法都要从低位开始。

因为：

```
1 进位和借位都是从低位影响高位。
```

例如：

```
1    98
2  +  7
3  ----
```

个位：

```
1 8 + 7 = 15
```

个位留下 5，向十位进 1。

所以必须先算个位，再算十位。

五、字符和数字的转换

字符串里的数字字符不能直接当数字用。

例如：

```
1 char c='7';
```

如果想得到数字 7：

```
1 int x=c-'0';
```

如果想把数字 7 变回字符：

```
1 char c=x+'0';
```

常见写法：

```
1 int x=s[i]-'0';
```

六、高精度加法

1. 题型

给两个很大的非负整数，求它们的和。

例如：

```
1 123456789123456789
```

2 987654321987654321

普通 `long long` 可能装不下，所以要用字符串。

2. 思路

高精度加法步骤：

1. 用 `string` 读入两个大整数
 2. 从两个字符串的末尾开始加
 3. 每次取一位相加，再加上进位
 4. 当前位结果是 `sum % 10`
 5. 新的进位是 `sum / 10`
 6. 最后如果还有进位，要补上
 7. 因为结果是从低位往高位得到的，所以最后要反转
-

3. 加法模板

```
1  #include<iostream>
2  #include<algorithm>
3  using namespace std;
4
5  string add(string a,string b)
6  {
7      string ans="";
8      int i=a.size()-1;
9      int j=b.size()-1;
10     int carry=0;
11
12     while(i>=0 || j>=0 || carry)
13     {
14         int x=0,y=0;
15
16         if(i>=0)
17         {
18             x=a[i]-'0';
19             i--;
20         }
21
22         if(j>=0)
```

```

23     {
24         y=b[j]-'0';
25         j--;
26     }
27
28     int sum=x+y+carry;
29
30     ans.push_back(sum%10+'0');
31     carry=sum/10;
32 }
33
34 reverse(ans.begin(),ans.end());
35
36 return ans;
37 }
38
39 int main()
40 {
41     string a,b;
42     cin>>a>>b;
43
44     cout<<add(a,b);
45
46     return 0;
47 }

```

4. 加法关键点

为什么 `while(i>=0 || j>=0 || carry)` ?

因为只要还有：

- 1 a 没加完
- 2 b 没加完
- 3 还有进位

就必须继续算。

为什么最后要 reverse?

因为计算过程是从低位到高位。

比如先得到个位，再得到十位，再得到百位。

所以 `ans` 一开始是反的，需要最后反转。

七、高精度减法

1. 题型

给两个大整数，求：

```
1  a - b
```

一般题目可能要求：

```
1  a >= b
```

也可能没有保证。

如果没有保证，就要先比较大小。

2. 思路

高精度减法步骤：

1. 用 `string` 读入两个大整数
 2. 判断谁大谁小
 3. 用大的减小的
 4. 从低位开始减
 5. 如果不够减，就向高位借 1
 6. 处理前导 0
 7. 如果原来是小数减大数，最后补负号
-

3. 比较两个大整数大小

不能直接转成整数。

应该这样比较：

```

1  bool smaller(string a,string b)
2  {
3      if(a.size()!=b.size())
4      {
5          return a.size()<b.size();
6      }
7
8      return a<b;
9  }

```

含义:

- 1 如果长度不同，长度短的数字小；
- 2 如果长度相同，按字典序比较。

4. 减法模板

```

1  #include<iostream>
2  #include<algorithm>
3  using namespace std;
4
5  bool smaller(string a,string b)
6  {
7      if(a.size()!=b.size())
8      {
9          return a.size()<b.size();
10     }
11
12     return a<b;
13 }
14
15 string sub(string a,string b)
16 {
17     string ans="";
18     int i=a.size()-1;
19     int j=b.size()-1;
20     int borrow=0;
21
22     while(i>=0 || j>=0)
23     {
24         int x=0,y=0;
25

```

```

26         if(i>=0)
27         {
28             x=a[i]-'0';
29             i--;
30         }
31
32         if(j>=0)
33         {
34             y=b[j]-'0';
35             j--;
36         }
37
38         x-=borrow;
39
40         if(x<y)
41         {
42             x+=10;
43             borrow=1;
44         }
45         else
46         {
47             borrow=0;
48         }
49
50         ans.push_back(x-y+'0');
51     }
52
53     while(ans.size()>1 && ans.back()=='0')
54     {
55         ans.pop_back();
56     }
57
58     reverse(ans.begin(),ans.end());
59
60     return ans;
61 }
62
63 int main()
64 {
65     string a,b;
66     cin>>a>>b;
67
68     if(smaller(a,b))
69     {
70         cout<<"-"<<sub(b,a);
71     }

```



```
72     else
73     {
74         cout<<sub(a,b);
75     }
76
77     return 0;
78 }
```

5. 减法关键点

为什么要去前导 0?

例如：

```
1  1000 - 999 = 0001
```

真正结果应该是：

```
1  1
```

因为 `ans` 此时是反着存的，所以低位在前，高位在后。

前导 0 在反向字符串里表现为：

```
1  ans.back()=='0'
```

所以用：

```
1  while(ans.size()>1 && ans.back()=='0')
2  {
3      ans.pop_back();
4  }
```

为什么要先比较大小?

因为如果：

```
1  123 - 456
```

结果是负数。

高精度减法模板通常只负责：

```
1  大数 - 小数
```

所以要先判断谁大。

八、高精度乘法

1. 题型

给两个大整数，求乘积。

例如：

```
1  123456789 × 987654321
```

结果可能非常大，不能用 `long long` 。

2. 思路

乘法本质也是模拟竖式。

例如：

```
1    123
2  ×   45
3  -----
4    615
5   492
6  -----
7   5535
```

程序中常用数组存每一位结果。

3. 为什么乘法常用 `vector`?

因为乘法中一位会影响多个位置。

例如：

```
1  a 的第 i 位 × b 的第 j 位
```

会累加到：

```
1  ans[i+j]
```

所以用数组或 vector 更方便。

4. 反向存储乘法模板

```
1  #include<iostream>
2  #include<vector>
3  #include<algorithm>
4  using namespace std;
5
6  string mul(string a,string b)
7  {
8      vector<int> A,B;
9
10     for(int i=a.size()-1;i>=0;i--)
11     {
12         A.push_back(a[i]-'0');
13     }
14
15     for(int i=b.size()-1;i>=0;i--)
16     {
17         B.push_back(b[i]-'0');
18     }
19
20     vector<int> C(A.size()+B.size()+5,0);
21
22     for(int i=0;i<A.size();i++)
23     {
24         for(int j=0;j<B.size();j++)
25         {
26             C[i+j]+=A[i]*B[j];
27         }
28     }
29
30     for(int i=0;i<C.size()-1;i++)
```

```

31     {
32         C[i+1]+=C[i]/10;
33         C[i]%=10;
34     }
35
36     while(C.size()>1 && C.back() == 0)
37     {
38         C.pop_back();
39     }
40
41     string ans="";
42
43     for(int i=C.size()-1;i>=0;i--)
44     {
45         ans.push_back(C[i]+'0');
46     }
47
48     return ans;
49 }
50
51 int main()
52 {
53     string a,b;
54     cin>>a>>b;
55
56     cout<<mul(a,b);
57
58     return 0;
59 }

```

5. 乘法关键点

为什么 $C[i+j] += A[i] * B[j]$?

因为反向存储后：

- 1 $A[i]$ 表示 10^i 位
- 2 $B[j]$ 表示 10^j 位

它们相乘后应该放在：

- 1 $10^{(i+j)}$ 位

所以累加到：

```
1 C[i+j]
```

为什么最后要统一处理进位？

因为很多位置可能被多次累加。

例如：

```
1 十位可能来自多个乘积
```

所以先全部累加，再统一进位：

```
1 for(int i=0;i<C.size()-1;i++)
2 {
3     C[i+1]+=C[i]/10;
4     C[i]%=10;
5 }
```

九、高精度阶乘

1. 题型

求：

```
1 n!
```

当 `n` 较大时，结果位数很多，不能用普通整数。

典型题：

```
1 P1009 阶乘之和
```

2. 思路

阶乘可以看成：

```
1  1 × 2 × 3 × ... × n
```

如果要算大数阶乘，只需要反复进行：

```
1  高精度 × 低精度
```

也就是：

```
1  大整数乘一个普通 int
```

3. 高精度乘低精度模板

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  vector<int> mul(vector<int> a,int b)
6  {
7      vector<int> c;
8      int carry=0;
9
10     for(int i=0;i<a.size();i++)
11     {
12         int t=a[i]*b+carry;
13         c.push_back(t%10);
14         carry=t/10;
15     }
16
17     while(carry)
18     {
19         c.push_back(carry%10);
20         carry/=10;
21     }
22
23     while(c.size()>1 && c.back()==0)
24     {
25         c.pop_back();
26     }
27
28     return c;
29 }
30
```

```
31  int main()
32  {
33      int n;
34      cin>>n;
35
36      vector<int> ans;
37      ans.push_back(1);
38
39      for(int i=2;i<=n;i++)
40      {
41          ans=mul(ans,i);
42      }
43
44      for(int i=ans.size()-1;i>=0;i--)
45      {
46          cout<<ans[i];
47      }
48
49      return 0;
50  }
```

十、高精度常见边角问题

1. 为什么喜欢倒序存？

因为低位在前，进位更方便。

比如数字：

```
1  12345
```

倒序存成：

```
1  5 4 3 2 1
```

这样：

```
1  第 0 位就是个位
2  第 1 位就是十位
3  第 2 位就是百位
```

计算时更自然。

2. 字符串正序，vector 倒序

常见习惯：

```
1  string 读入时是正序
2  vector<int> 存计算时改成倒序
```

例如：

```
1  string s;
2  cin>>s;
3
4  vector<int> a;
5
6  for(int i=s.size()-1;i>=0;i--)
7  {
8      a.push_back(s[i]-'0');
9  }
```

3. 去前导 0

正序字符串中：

```
1  000123
```

前导 0 在前面。

但是倒序 vector 中：

```
1  3 2 1 0 0 0
```

前导 0 在后面。

所以常用：

```
1  while(a.size()>1 && a.back()==0)
2  {
3      a.pop_back();
4  }
```

4. 结果为 0 的情况

一定要保留一个 0。

不能把所有 0 都删完。

所以条件要写：

```
1 while(a.size()>1 && a.back()==0)
```

而不是：

```
1 while(a.back()==0)
```

5. char 和 int 不能混用

错误：

```
1 ans.push_back(sum%10);
```

如果 `ans` 是 `string`，这样不对。

正确：

```
1 ans.push_back(sum%10+'0');
```

6. reverse 的作用

如果用 `string` 存答案，计算时从低位往高位加：

```
1 ans.push_back(...)
```

那么答案是反的。

最后要：

```
1 reverse(ans.begin(),ans.end());
```

十一、常见错误总结

1. 忘记最后还有进位

例如：

```
1  999 + 1
```

最后应该得到：

```
1  1000
```

所以循环条件要包含：

```
1  carry
```

2. 减法忘记处理借位

如果：

```
1  1000 - 1
```

中间会连续借位。

必须正确维护：

```
1  borrow
```

3. 减法没有先比较大小

如果题目没有保证 `a >= b`，必须先比较。

4. 前导 0 没处理

例如结果变成：

```
1  000123
```

要处理成：

```
1 123
```

5. 字符没有减 '0'

错误：

```
1 int x=s[i];
```

正确：

```
1 int x=s[i]-'0';
```

十二、本专题做过的题

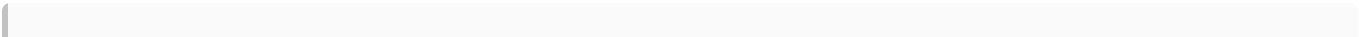
题号	题目	类型	题面简述	对应知识点
P1601	A+B Problem (高精)	高精度加法	输入两个很大的非负整数，输出它们的和	字符串加法、进位
P2142	高精度减法	高精度减法	输入两个很大的非负整数，输出它们的差	比较大小、借位、负号
P1303	A*B Problem	高精度乘法	输入两个很大的非负整数，输出它们的乘积	竖式乘法、进位
P1009	阶乘之和	高精度乘法 / 加法	求 $1!+2!+\dots+n!$	高精度乘低精度、高精度加法

十三、本章最重要的结论

高精度的本质不是背代码，而是：

```
1 用数组或字符串模拟竖式计算。
```

遇到高精度题时，先判断：



- 1 是加法、减法、乘法，还是阶乘类反复乘？

然后抓住三个核心：

- 1 低位开始
- 2 进位 / 借位
- 3 去前导 0

只要这三个点稳，高精度基础题就不会乱。